Trabajo Práctico Obligatorio Integrador

Paradigma Orientado a Objetos

Clínica Veterinaria - Informe Final

Integrantes:

1. Melina Bovarines - N° de legajo: 1226714
2. Giovanna Manograsso - N° de legajo: 1230393
3. Manuela Torres - N° de legajo: 1220169

Profesor: Maria Angela Leon

Fecha de entrega: 07/11/26

Cuatrimestre: 2° Año: 2026

UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

Cambios respecto del diseño original en el diagrama de clases

El diseño original se centraba en la estructura de datos del consultorio veterinario. Durante el desarrollo se incorporó la lógica de negocio y se corrigieron inconsistencias de tipos y responsabilidades. Los cambios se agrupan en seis categorías. La incorporación de las clases de servicio mejoró la separación de responsabilidades, y la corrección de tipos dio mayor consistencia al diagrama.

1. Incorporación de clases de servicio

El diseño original tenía pocas clases de servicio (ServicioStock y ServicioRecordatorios), y con operaciones mínimas. Se agregaron:

- ServicioTurnos: centraliza la asignación, reprogramación y cancelación de turnos, el registro de ausencias y atenciones, la verificación de seña y la asignación de urgencias.
- RegistroDuenoMascotas: gestiona el alta de dueños y mascotas y la transferencia de una mascota a un nuevo dueño con su fecha, lo que alimenta el HistorialDueno.

Además, ServicioStock se amplió con registrarProducto, ingresarLote y hayStock. Se eliminó descontarPorVencimiento(), porque el vencimiento pasó a resolverse en el propio lote (LoteProducto.estaVencido()).

2. Nuevas clases del dominio

- Clase nueva: Conejo (y CONEJO en Especie).
  Motivo: Se encontraba en la consigna y no había sido implementado.
- Clase nueva: NotaRectificatoria
  Motivo: Permitir corregir una consulta ya cerrada sin modificar el registro original, dejando constancia del texto, la fecha y el veterinario firmante.
- Clase Nueva: InsumoUtilizado
  Motivo: Representar de forma explícita el consumo de insumos en una consulta, reemplazando al método registrarUsoInsumo.

3. Atributos agregados

- **Dueno**: planSalud, para calcular descuentos en la facturación.
- **Veterinario**: deGuardia, necesario para la asignación de urgencias.
- **Turno**: número (identificación y búsqueda) y cargoCancelacion (registro del cargo por cancelación tardía).
- **Consulta**: cerrada (bloquea modificaciones tras el cierre) y tipoAtencion.
- **Medicamento**: principioActivo y presentacion.
- **Insumo**: unidadMedida.
- **LoteProducto**: costoUnitario, para valorizar el stock por lote.
- **Proveedor**: email.
- **ReporteVacunasProximas**: fecha, como fecha de referencia del reporte.

4. Atributos modificados o reubicados:

- **Turno**: fecha y hora se unificaron en fechaHora (LocalDateTime), lo que simplifica el cálculo de disponibilidad y superposición.
- **Pago.cantidadCuotas** paso a ser: **Credito.cantidadCuotas** las cuotas solo tienen sentido para el pago con crédito, por lo que pasan a la clase que representa ese medio de pago.
- **UsoInsumo.precioUnitario** se eliminó: el costo se obtiene del lote, evitando duplicar información.

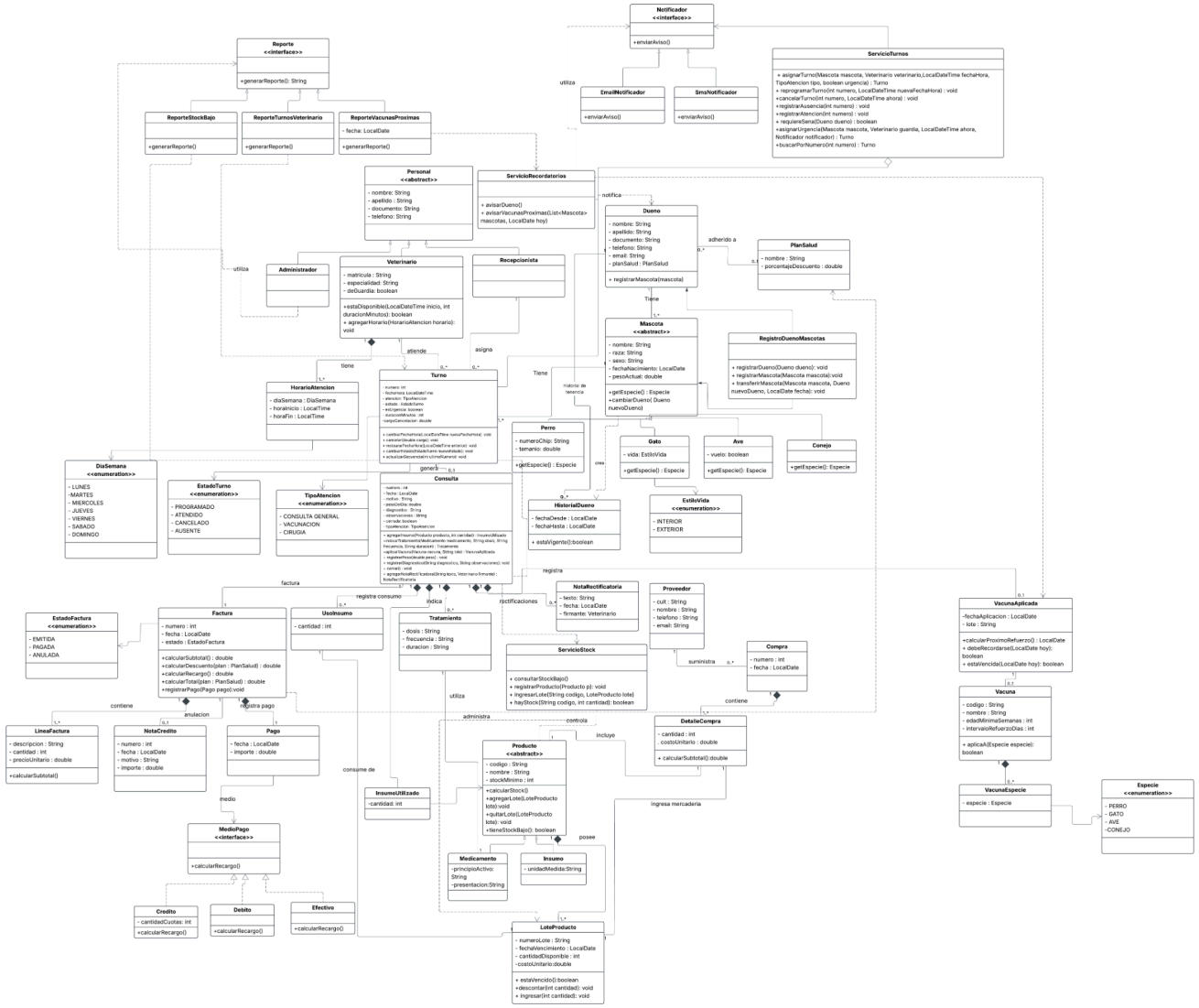
- **Vacuna.edadMinima** paso a ser: **edadMinimaSemanas** se explicita la unidad de medida.
- **Perro.Tamaño** paso a ser **tamanio**: se evitan caracteres especiales y se respeta la convención de nombres.

5. Comportamiento agregado a las clases existentes:

- **Producto**: agregarLote, quitarLote, tieneStockBajo.
- **LoteProducto**: estaVencido, descontar, ingresar.
- **Turno**:cambiarFechaHora,cancelar, restaurarFechaHora, cambiarEstado, actualizarSecuencia.
- **Consulta**:agregarInsumo,registrarPeso,registrarDiagnostico,cerrar, agregarNotaRectificadora.
- **Veterinario**: agregarHorario, y estaDisponible pasó de (fecha, hora) a (LocalDateTime inicio, int duracionMinutos), considerando la duración del turno.
- **Vacuna**: aplicaA(Especie).
- **VacunaAplicada**: debeRecordarse y estaVencida.
- **Factura**: registrarPago.
- **HistorialDueno**: estaVigente.
- **DetalleCompra**: calcularSubtotal.
- **Dueno**: registrarMascota.
- **ServicioRecordatorios**: avisarVacunasProximas.

6**. Mejoras de consistencia:**

- Los tipos Date se reemplazaron por LocalDate, LocalDateTime o LocalTime, y las horas de HorarioAtencion, antes int, pasaron a LocalTime.
- Los montos y pesos pasaron de float a double, evitando errores de precisión en los cálculos de facturación.
- telefono pasó de int a String, ya que puede contener ceros iniciales, prefijos o símbolos.
- Se agregaron tipos explícitos de parámetros y de retorno en los métodos (por ejemplo, Mascota.cambiarDueno(Dueno nuevoDueno) y Reporte.generarReporte(): String).



link: https://lucid.app/lucidchart/81295d27-a414-4167-be68-2fdf4566be32/view

Modificaciones realizadas en el diagrama de casos de uso

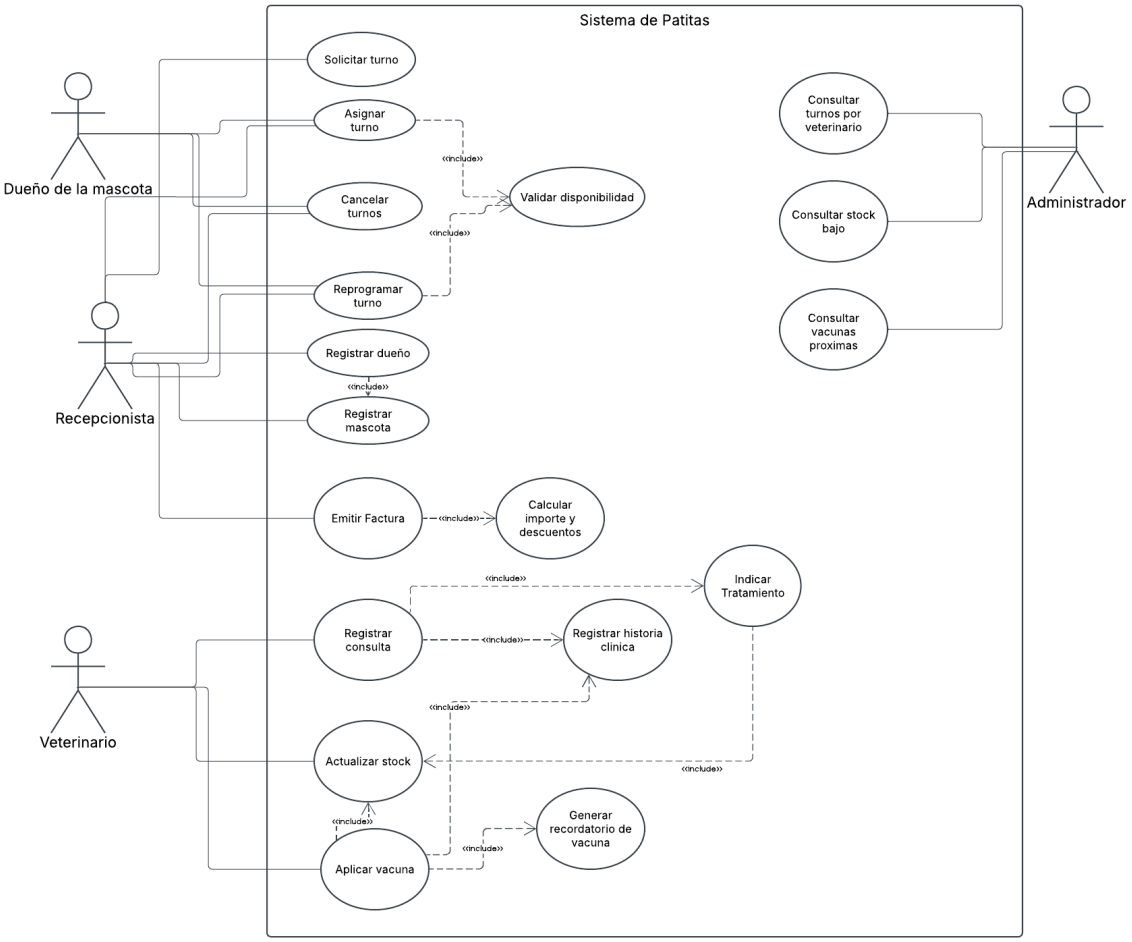
En la versión final del diagrama de casos de uso del Sistema de Patitas se realizaron modificaciones con el objetivo de representar de manera más completa e integrada las funcionalidades del sistema.

Se incorporaron nuevas relaciones de tipo <<include>>, que permiten mostrar las acciones que deben ejecutarse obligatoriamente como parte de un caso de uso principal. Entre ellas, se agregó la validación de disponibilidad en la asignación y reprogramación de turnos, la relación entre el registro del dueño y el registro de su mascota, y la incorporación del registro de historia clínica y la indicación de tratamientos dentro de la consulta veterinaria.

También se ampliaron las funcionalidades relacionadas con la aplicación de vacunas, incorporando la actualización del stock, el registro de la vacunación en la historia clínica y la generación de recordatorios para futuras dosis.

Estas modificaciones permiten representar una mayor conexión entre las distintas funcionalidades del sistema, evitando que los casos de uso queden aislados y mostrando cómo se complementan entre sí para cumplir con los procesos de la clínica veterinaria.

Por último, se mantuvieron los actores y las funcionalidades principales de la versión anterior, pero se mejoró la representación de las relaciones entre ellos y los casos de uso, logrando un diagrama más completo y detallado.



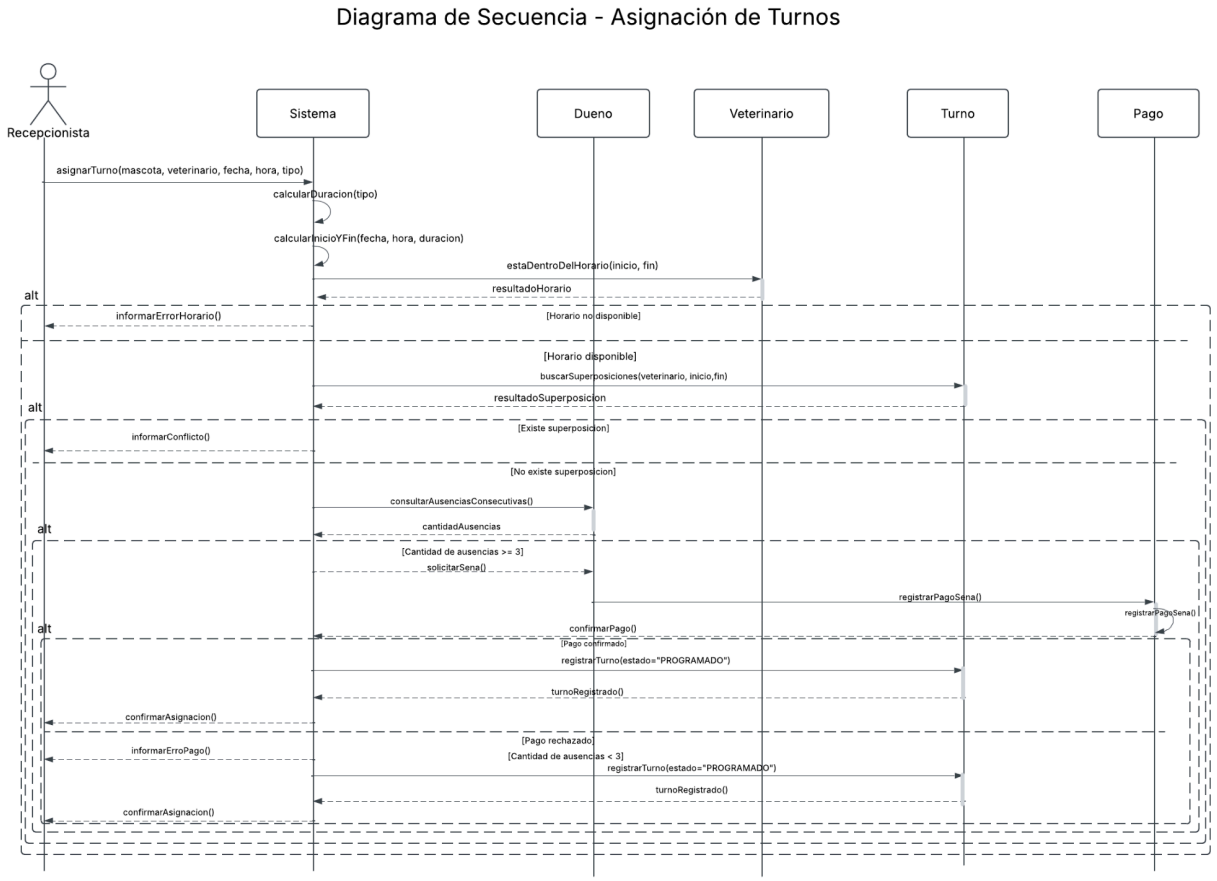
link: https://lucid.app/lucidchart/0bf1c07c-a11a-4428-a903-3887a88b654e/view

Modificaciones del diagrama de secuencia asignar turno

Las modificaciones realizadas en el diagrama de secuencia permitieron representar de manera más detallada el proceso de asignación de turnos veterinarios. Se agregó el cálculo del horario de inicio y finalización del turno a partir de su duración, para luego verificar que se encuentre dentro del horario de atención. También se incorporó la búsqueda de superposiciones con otros turnos del mismo veterinario, evitando asignar dos turnos en el mismo horario.

Por otro lado, se especificó el control de las ausencias consecutivas del dueño. Cuando la cantidad de ausencias es igual o superior a tres, se solicita el pago de una seña antes de continuar con la asignación. Además, se incorporó una alternativa para contemplar tanto la confirmación como el rechazo del pago, informando el error cuando no se concreta. Si el pago es confirmado, o si el dueño tiene menos de tres ausencias consecutivas, el sistema registra el turno con estado "PROGRAMADO" y confirma la asignación al recepcionista.

De esta manera, el diagrama final contempla las distintas condiciones que pueden presentarse durante el proceso y establece con mayor precisión las acciones y responsabilidades de cada participante.



link: https://lucid.app/lucidchart/92f49756-9819-4f3a-9d21-9f32191880e2/view

Plan de pruebas funcionales

Plan de pruebas funcionales

Dificultades encontradas

Durante el desarrollo del proyecto nos encontramos con distintas dificultades, tanto a nivel técnico como organizativo, que tuvimos que resolver para lograr el funcionamiento integrado del sistema.

1. Organización e integración del trabajo en equipo

Una de las principales dificultades fue trabajar de manera simultánea en distintas ramas de GitHub. En algunos momentos surgieron inconvenientes al realizar los merge, traer los cambios de otros integrantes o integrar clases que dependían de funcionalidades desarrolladas por otros compañeros. Esto requirió revisar los cambios, resolver conflictos y coordinar la integración para evitar perder código o generar inconsistencias.

2. Relación entre las clases y distribución de responsabilidades

Al implementar las funcionalidades, surgieron dudas sobre qué clase debía encargarse de determinadas operaciones. Por ejemplo, la asignación de turnos, el control del stock y la facturación involucraban varias clases relacionadas entre sí. Tuvimos que revisar el diseño original y reorganizar algunas responsabilidades, incorporando clases de servicio para evitar concentrar demasiada lógica en una sola clase.

3. Implementación de las reglas de negocio
Otra dificultad importante fue trasladar las reglas de negocio de la consigna al código Java. Algunas funcionalidades requerían contemplar distintas condiciones, como el control de tres ausencias consecutivas, el cobro de señas, las cancelaciones tardías, las urgencias y la validación de disponibilidad de los veterinarios. Fue necesario analizar cada situación y contemplar tanto los casos correctos como los posibles errores.

4. Manejo de fechas, horarios y validaciones
Tuvimos dificultades al trabajar con fechas y horarios, especialmente al calcular la duración de los turnos, verificar que se encuentren dentro del horario de atención y evitar superposiciones. También fue necesario contemplar situaciones particulares, como los días en los que no hay atención y las urgencias que requieren reorganizar otros turnos.

5. Persistencia de datos mediante archivos
La implementación de la persistencia también presentó dificultades, ya que debíamos lograr que la información continuara disponible después de cerrar el programa. Además, tuvimos que contemplar situaciones en las que los archivos estuvieran dañados o no pudieran leerse correctamente, evitando que estos inconvenientes impidieran iniciar el sistema.

6. Pruebas e integración final
Por último, una dificultad fue comprobar que todas las funcionalidades funcionaran correctamente en conjunto. Algunas operaciones dependían de otras, por lo que no alcanzaba con verificar cada método de manera individual. Por ejemplo, al confirmar una consulta debíamos controlar el stock, cerrar la atención y generar la factura. Esto nos llevó a realizar pruebas unitarias y funcionales, identificar errores y efectuar correcciones hasta conseguir un funcionamiento más consistente.
En general, estas dificultades nos permitieron comprender la importancia de planificar el desarrollo, mantener una buena comunicación entre los integrantes y probar las funcionalidades durante todo el proceso, en lugar de dejar las verificaciones únicamente para el final.

Distribución y organización de las tareas
Para llevar adelante el desarrollo del sistema, organizamos el trabajo utilizando Git y GitHub, dividiendo las funcionalidades en distintas ramas (*branches*) para facilitar el trabajo en equipo y permitir que cada integrante pudiera desarrollar una parte específica del proyecto de manera independiente.
En primer lugar, creamos la rama principal main, donde se integraron las versiones finales y revisadas del sistema. Además, creamos una rama feature-documentacion, destinada a desarrollar y almacenar la documentación inicial del proyecto, incluyendo los diagramas UML y las reglas de negocio.
Posteriormente, dividimos el desarrollo en tres ramas principales, cada una encargada de un conjunto de funcionalidades:

1. Rama feature/duenos-mascotas
Esta rama estuvo destinada al desarrollo de las clases y funcionalidades relacionadas con los dueños y sus mascotas. Entre las principales tareas realizadas se encuentran:

- Creación de las clases Dueno, Mascota, Perro, Gato, Ave e HistorialDueno.
- Implementación del registro y consulta de dueños y mascotas.
- Desarrollo de la funcionalidad de cambio de dueño, conservando el historial clínico de la mascota.

- Desarrollo de las clases relacionadas con las consultas clínicas, los tratamientos y las vacunas aplicadas.
- Implementación de validaciones correspondientes a la especie, la edad mínima y el peso de las mascotas.

2. Rama feature/turnos-consultas

Esta rama se enfocó en la gestión del personal veterinario, los turnos y las consultas médicas. Las tareas principales fueron:

- Creación de las clases Personal, Veterinario y Turno, junto con la definición de los horarios de atención.
- Implementación de las funcionalidades para asignar, reprogramar y cancelar turnos.
- Validación de la duración de los turnos, los horarios de atención y la existencia de superposiciones.
- Control de ausencias consecutivas de los clientes y de las señas correspondientes.
- Registro de las atenciones veterinarias y de los tratamientos realizados.

3. Rama feature/stock-facturacion-reportes

Esta rama estuvo orientada a la administración del stock, la facturación y la generación de reportes del sistema. Se desarrollaron las siguientes funcionalidades:

- Creación de las clases Producto, LoteProducto, UsoInsumo y ServicioStock.
- Implementación del sistema de facturación, incluyendo facturas, líneas de factura y medios de pago.
- Desarrollo de reportes relacionados con los turnos, el stock bajo y las vacunas próximas a vencer o aplicarse.
- Implementación de recordatorios y de las interfaces Reporte y Notificador.
- Desarrollo de la persistencia de datos mediante archivos.
- Integración de las funcionalidades necesarias para el funcionamiento general del sistema.

Una vez finalizado el desarrollo de estas partes, avanzamos con la integración de las distintas ramas para unificar las funcionalidades implementadas. En esta etapa, desarrollamos el menú principal dentro de la carpeta vista, con el objetivo de facilitar la interacción y permitir probar las distintas opciones del sistema.

Posteriormente, trabajamos en el controlador, encargado de conectar la interfaz con las clases del modelo y gestionar las operaciones y la persistencia de los datos.

Por último, realizamos las pruebas unitarias de las distintas funcionalidades y elaboramos el plan de pruebas para verificar el correcto funcionamiento del sistema. A partir de los resultados obtenidos, identificamos y corregimos los errores e inconvenientes que fueron surgiendo durante el proceso, realizando los ajustes necesarios hasta lograr una integración y un funcionamiento adecuado de los diferentes módulos.

De esta manera, el desarrollo se llevó a cabo de forma organizada y colaborativa, permitiendo distribuir las responsabilidades, trabajar en paralelo y finalmente integrar todas las funcionalidades en un único sistema.

Reflexión del grupo

La realización de este proyecto fue una experiencia que nos permitió poner en práctica los conocimientos adquiridos durante la materia de Programación Orientada a Objetos. A lo largo del desarrollo tuvimos que enfrentarnos a diferentes desafíos relacionados con el diseño de clases, la implementación de reglas de negocio, el manejo de archivos, el uso de Git y la integración de las distintas partes del sistema.

Si bien fue un trabajo que presentó varias dificultades y que en algunos momentos nos resultó complejo, nos permitió comprender que desarrollar un sistema no consiste solamente en escribir código, sino también en analizar un problema, organizar las tareas, definir responsabilidades y lograr que todos los componentes funcionen correctamente en conjunto.

Uno de los aprendizajes más importantes fue trabajar de manera colaborativa utilizando Git y GitHub, ya que tuvimos que organizarnos, dividir las funcionalidades y aprender a integrar los cambios realizados por cada integrante. También comprendimos la importancia de realizar pruebas para detectar errores y verificar que el sistema respete las reglas de negocio establecidas.

Consideramos que el proyecto nos ayudó a identificar los temas que todavía necesitamos reforzar, especialmente en relación con la organización del código, la lógica de programación y la resolución de errores. A pesar de las dificultades, logramos desarrollar un sistema que reúne las principales funcionalidades solicitadas y que nos permitió acercarnos a una situación más real de desarrollo de software.

Como conclusión, nos llevamos una experiencia de aprendizaje que nos permitió reconocer tanto nuestras fortalezas como los aspectos que debemos seguir mejorando para futuros proyectos.